



PORTFOLIO EDITION

BLUE TEAM INTERNSHIP

Week 1 Task Report

SOC Foundations and Lab Setup

Prepared by Malaika Azhar · Week 1 (Days 1–7)

Table of Contents

Hardware & Setup Note	3
Executive Summary	4
Tools and Technologies Used	5
Day 1 — Networking Fundamentals for SOC	6
Day 2 — VM Lab Setup	11
Day 3-4 — Wazuh Manager Setup	15
Day 5-6 — Agent Deployment and Alert Testing	17
Day 7 — Dashboard Navigation and Alert Triage	21
Common Issues — What Actually Happened This Week	24
Closing Reflection — Week 1	25

Hardware & Setup Note

My laptop (HP 255 G5, AMD A6-7310, 4GB RAM, 128GB SSD) does not meet the brief's minimum spec for running 3 VMs simultaneously (4GB+ RAM each). After raising this with program support and discussing it directly with you, the following interim plan was approved:

Plan Item	Approved Decision
Build Scope	Build one VM only: Ubuntu Server, allocated 2GB RAM.
Windows Agent	Use host Windows as the Wazuh agent instead of a separate Windows 10 VM.
Kali Linux VM	Deferred until a RAM upgrade is complete.
Hypervisor	VMware Workstation (brief specifies VirtualBox; confirming acceptability).

All deviations from the original brief in this report follow directly from this approved plan.

Executive Summary

This report documents Week 1 of the Blue Team Internship: building a working SOC lab from scratch and developing a practical understanding of network traffic and SIEM architecture, end to end.

Day	What Happened
Day 1	Studied core networking concepts (OSI/TCP-IP, subnetting, ports, protocols) and captured live traffic in Wireshark, identifying 5 protocols with normal vs suspicious traffic patterns.
Day 2	Built a VMware-based Ubuntu Server VM with dual network adapters (NAT + Host-only), confirmed connectivity, and took a rollback snapshot.
Day 3-4	Attempted local Wazuh installation, hit a genuine hardware ceiling on 2GB RAM, and pivoted to Wazuh Cloud's trial as a documented, deliberate substitution.
Day 5-6	Deployed two agents (Ubuntu VM + Windows host), confirmed both Active simultaneously, and generated a real, verifiable failed-login alert end to end.
Day 7	Triaged 10 real alert types pulled directly from the Events view, assessing each as routine or worth investigating, rather than relying on severity level alone.

Every screenshot in this report is from my own environment. Where the original brief's exact setup wasn't possible on my hardware, the report states what changed and why, rather than presenting a result that didn't actually happen.

Tools and Technologies Used

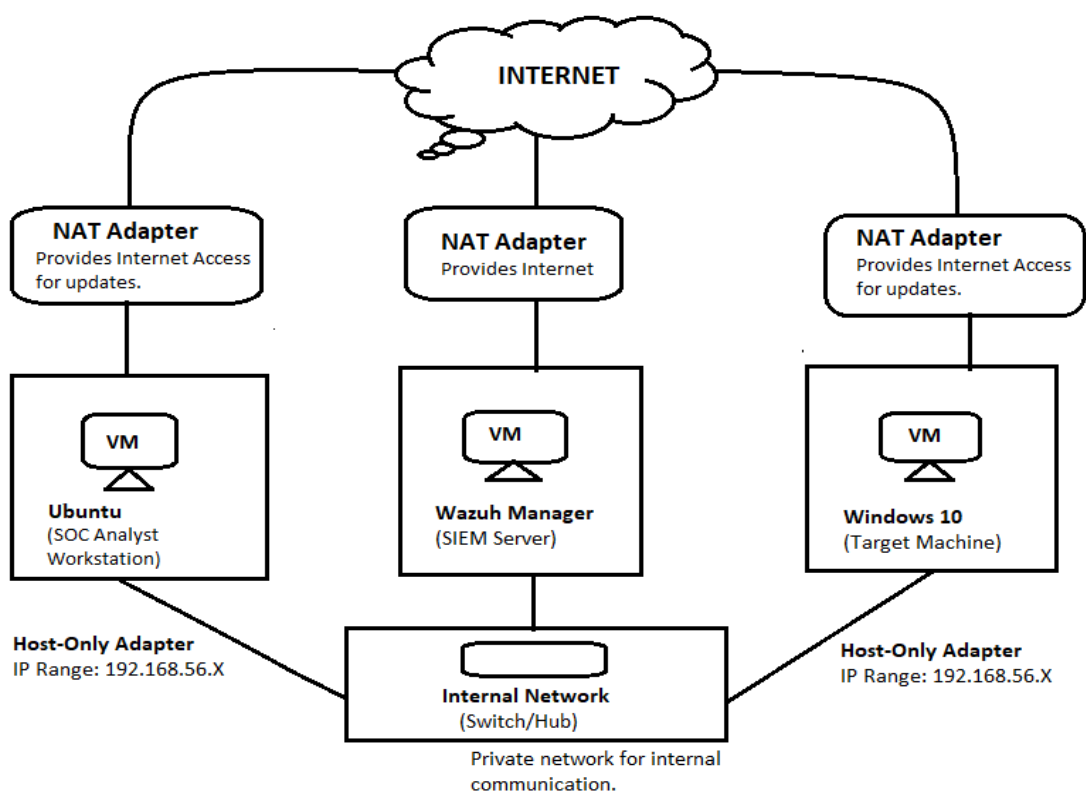
Tool / Technology	Purpose
VMware Workstation	Local VM hypervisor for the Ubuntu Server VM
Ubuntu Server 22.04.5 LTS	Monitored endpoint / local lab machine
Wireshark	Live traffic capture and protocol analysis
Wazuh Cloud (v4.14.5)	SIEM Manager, Indexer, and Dashboard (cloud-hosted, in place of a local install)
Wazuh Agent	Deployed on both the Ubuntu VM and the Windows host
Windows PowerShell	Windows agent installation
MITRE ATT&CK Framework	Mapping detected activity (e.g. Password Guessing) to standard attacker technique categories

Day 1 — Networking Fundamentals for SOC

1. Network Diagram — Planned VirtualBox Lab Topology

My lab uses three VMs: Ubuntu Analyst, Wazuh SIEM Server, and Windows 10 Target. Each VM has two adapters:

- NAT Adapter — connects each VM to the internet through my host machine, needed to download tools, security packages, and updates.
- Host-Only Adapter — an isolated internal network so the three VMs can talk to each other and exchange logs, without touching my real home network.



2. Tracing a Web Request — OSI and TCP/IP Models

Layer	What Happens
Application Layer	I type the URL and my browser sends an HTTP/HTTPS request. First, a DNS query goes out to find the server's IP address.

Layer	What Happens
Transport Layer	The request is split into TCP segments, targeting port 443 for secure traffic. TCP uses sequence numbers and handshakes to make sure no data is lost.
Network Layer	The source IP (my PC) and destination IP (the server) are added, forming an IP packet so routers know where to send it.
Data Link & Physical Layers	The packet gets a MAC address label and travels over Wi-Fi or cable as electronic signals. The server reverses this process to read the request.

3. Practical Subnetting and CIDR

A subnet mask tells a device which part of an IP address is the network and which part is the specific device. In a standard /24 network (subnet mask 255.255.255.0), the first three numbers represent the network ID.

Example: 192.168.1.5 and 192.168.1.20 share the same network part on a /24, so they can talk directly on the local switch without going through a router.

4. SOC Ports to Know

Port	Service	Use
22	SSH	Secure remote command-line access
25	SMTP	Routing email between servers
53	DNS	Matching domain names to IP addresses
80 / 8080	HTTP / alt-HTTP	Plain web traffic
443	HTTPS	Encrypted web browsing
445	SMB	Local file sharing and network printers
3389	RDP	Remote graphical access to Windows desktops

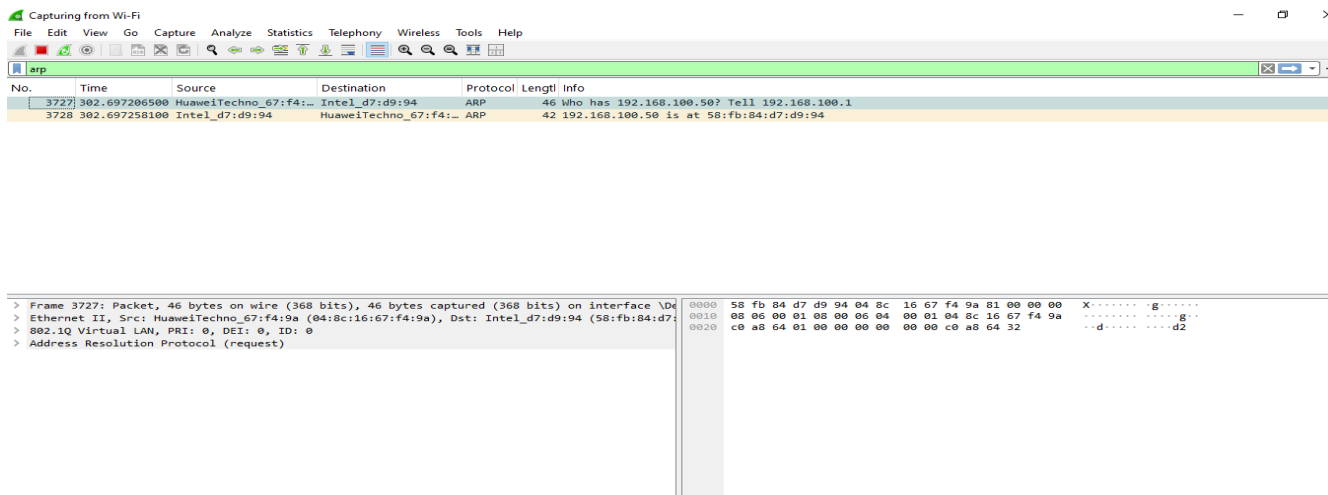
5. Protocols — Normal vs Suspicious Traffic

ARP

What it does	Helps a machine find a hardware MAC address when it only knows the IP — sends a local broadcast asking “who has this IP address?”
Normal traffic	My capture showed a normal exchange where my router asked “Who has 192.168.100.50? Tell 192.168.100.1” and my laptop replied with its MAC address — the router already knew my IP and was resolving it to a MAC address to deliver data.

Suspicious traffic

It's a red flag if one MAC address suddenly claims multiple different IP addresses — a sign of ARP spoofing or a Man-in-the-Middle (MITM) attack.



DNS

What it does

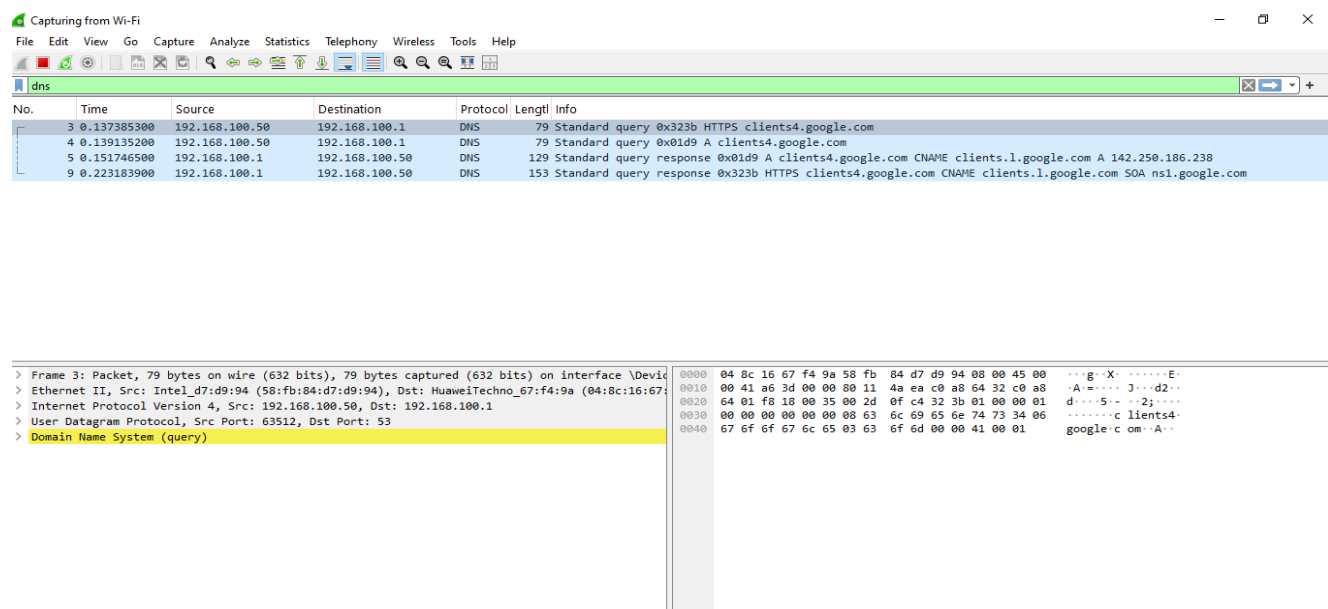
Works like a phonebook, turning website names into IP numbers.

Normal traffic

My capture showed standard queries for clients4.google.com, resolved via a CNAME to clients.l.google.com and then to an IP — ordinary Google service traffic.

Suspicious traffic

Long, randomized domain names usually mean malware trying to contact a Command and Control (C2) server. DNS traffic going to an unauthorized external server instead of the normal gateway is also a red flag.



HTTP

What it does	The basic protocol used to load websites and web files.
Normal traffic	My capture showed plain-text GET requests fetching Certificate Revocation Lists (.crl files) — routine background checks that happen automatically as part of certificate validation.
Suspicious traffic	HTTP doesn't encrypt anything — if logins, cookies, or sensitive data are sent over plain HTTP, anyone sniffing the network can read it.

Capturing from Wi-Fi

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

http

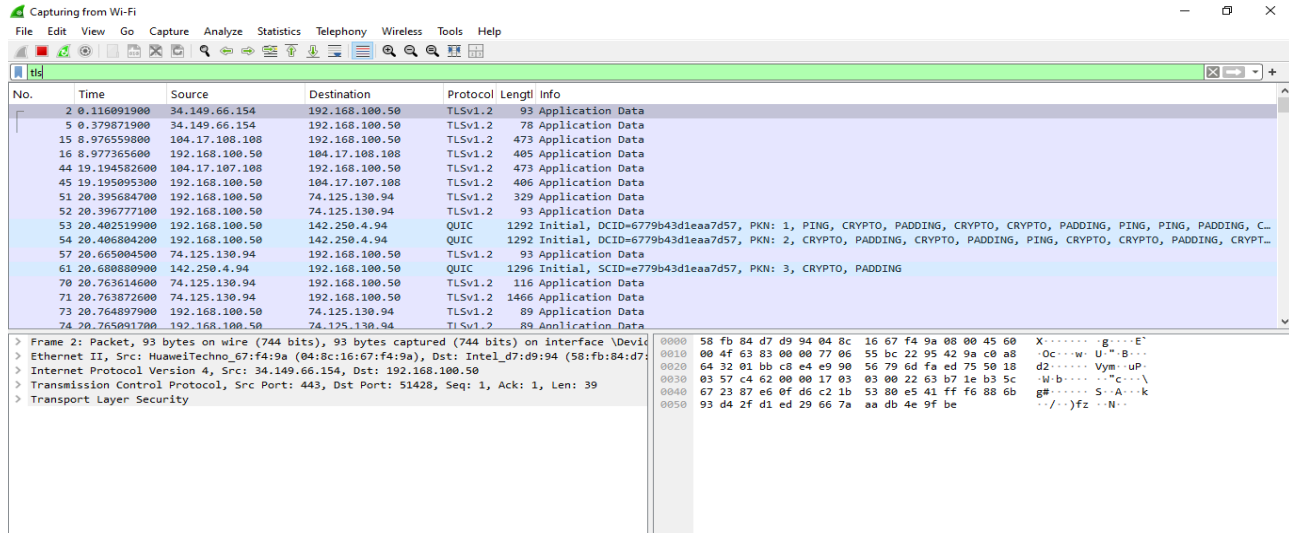
No.	Time	Source	Destination	Protocol	Length	Info
4121	324.998781200	192.168.100.50	142.251.38.3	HTTP	254	GET /r/r1.crl HTTP/1.1
4124	325.101303700	142.251.38.3	192.168.100.50	HTTP	276	HTTP/1.1 304 Not Modified
4144	325.595575600	192.168.100.50	92.122.6.109	HTTP	281	GET / HTTP/1.1
4148	325.708074000	92.122.6.109	192.168.100.50	HTTP	317	HTTP/1.1 304 Not Modified
4177	326.041283500	192.168.100.50	92.122.6.109	HTTP	281	GET / HTTP/1.1
4186	326.151028100	92.122.6.109	192.168.100.50	HTTP	317	HTTP/1.1 304 Not Modified
4300	327.020239300	192.168.100.50	23.11.39.161	HTTP	328	GET /DigiCertGlobalG2TLRSASHA2562020CA1-1.crl HTTP/1.1
4310	327.177083400	23.11.39.161	192.168.100.50	HTTP	288	HTTP/1.1 304 Not Modified
4333	327.829800500	192.168.100.50	2.16.158.81	HTTP	281	GET / HTTP/1.1
4335	327.940439300	2.16.158.81	192.168.100.50	HTTP	317	HTTP/1.1 304 Not Modified
4336	327.968022600	192.168.100.50	142.251.38.3	HTTP	265	GET /wr2/GSyTIN4PBrg.crl HTTP/1.1
4343	328.479681800	142.251.38.3	192.168.100.50	HTTP	277	HTTP/1.1 304 Not Modified
4344	328.500734900	192.168.100.50	23.11.39.161	HTTP	311	GET /DigiCertAssuredIDRootCA.crl HTTP/1.1
4347	328.639515800	23.11.39.161	192.168.100.50	HTTP	284	HTTP/1.1 304 Not Modified
4348	328.660760900	192.168.100.50	142.251.38.3	HTTP	265	GET /wr2/oBFYVahzgVI.crl HTTP/1.1
4364	328.767666700	142.251.38.3	192.168.100.50	PKTX-C	148	Certificate Revocation List

> Frame 4121: Packet, 254 bytes on wire (2032 bits), 254 bytes captured (2032 bits) on interface
 > Ethernet II, Src: Intel_d7:d9:94 (58:fb:84:d7:d9:94), Dst: HuaweiTechno_67:f4:9a (04:8c:16:67:f4:9a)
 > Internet Protocol Version 4, Src: 192.168.100.50, Dst: 142.251.38.3
 > Transmission Control Protocol, Src Port: 51506, Dst Port: 80, Seq: 1, Ack: 1, Len: 200
 > Hypertext Transfer Protocol

0000 04 8c 16 67 f4 9a 58 fb 84 d7 d9 94 08 00 45 00 ...g-X-.....E-
 0010 00 f0 78 cf 40 00 80 06 a7 5f c0 a8 64 32 8e fb ...x@..._..d2..
 0020 26 03 c9 32 00 50 00 d6 b0 9d 34 7e 15 78 50 18 &..2.P...4wxP..
 0030 01 03 22 08 00 00 47 45 54 20 2f 72 2f 72 31 2e ...GE T /r/r1..
 0040 63 72 6c 20 48 54 54 50 2f 31 2e 31 0d 0a 43 61 ..rl HTTP /1.1 ..Ca
 0050 63 68 65 2d 43 6f 6e 74 72 6f 6c 3a 20 6d 61 78 che-Cont rol: max
 0060 2d 61 67 65 20 3d 20 33 30 30 30 0d 0a 43 6f 6e -age = 3 000 ..Con
 0070 6e 65 63 74 69 6f 6e 3a 20 4b 65 65 70 2d 41 6c nnection: Keep-Al
 0080 69 76 65 0d 0a 41 63 63 65 70 74 3a 20 2a 2f 2a ive ..Acc ept: /*
 0090 0d 0a 49 66 2d 4d 6f 64 69 66 69 65 64 2d 53 69 ..If-Mod ified-Si
 00a0 6e 63 65 3a 20 57 65 64 2c 20 31 31 20 46 65 62 nce: Wed , 11 Feb
 00b0 20 32 30 32 36 20 31 34 3a 33 38 3a 30 30 20 47 2026 14 :38:00 G
 00c0 4d 54 0d 0a 55 73 65 72 2d 41 67 65 6e 74 3a 20 MT ..User ..Agent:
 00d0 4d 69 63 72 6f 73 6f 66 74 2d 43 72 79 70 74 6f Microsof t-Crypto
 00e0 41 50 49 2f 31 30 2e 30 0d 0a 48 6f 73 74 3a 20 API/10.0 ..Host:
 00f0 63 2e 70 6b 69 2e 67 6f 6f 67 0d 0a 0d 0a c.pki.go og...

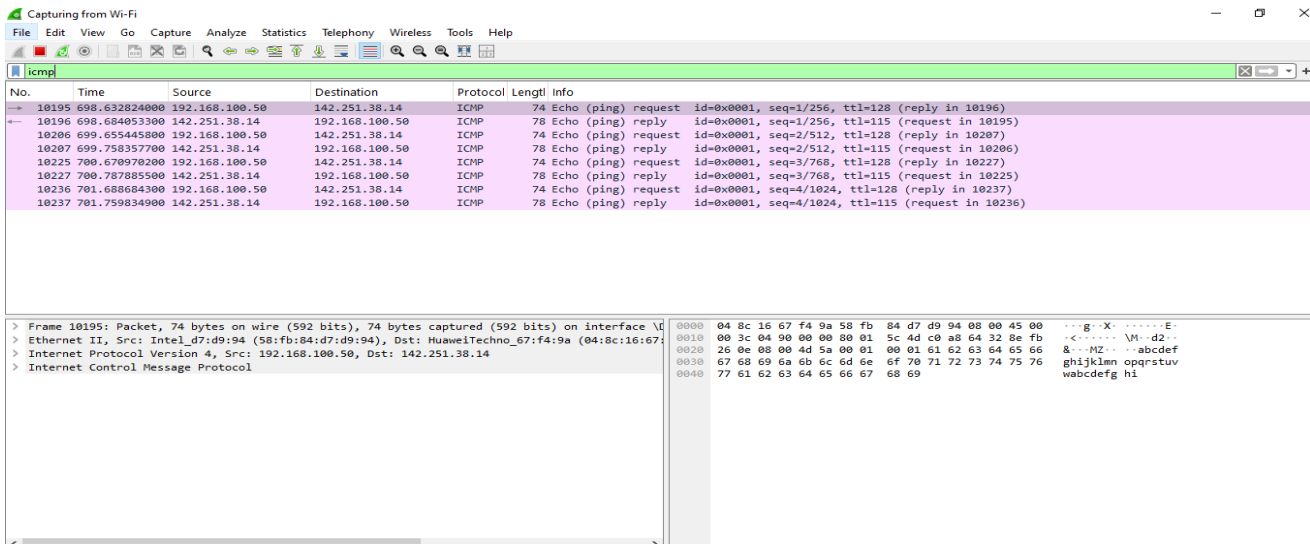
HTTPS / TLS

What it does	The encrypted version of HTTP. In Wireshark this traffic is labelled TLS.
Normal traffic	My capture showed multiple TLSv1.2 “Application Data” packets between my laptop and several remote servers — normal encrypted browsing where the content can't be read by anyone observing the traffic.
Suspicious traffic	An untrusted self-signed certificate, an expired cert, or a machine suddenly uploading large amounts of encrypted data to an unknown destination can indicate a data breach.



ICMP

What it does	A troubleshooting protocol used by the “ping” command to check if a remote system is reachable.
Normal traffic	My capture showed 4 echo request/reply pairs between my laptop and a Google IP — a normal reachability check, generated manually using the ping command since it didn't appear from passive browsing.
Suspicious traffic	A large flood of continuous or oversized ICMP packets usually means network scanning or a Denial-of-Service (DoS) attempt.



6. What I Learned — Day 1

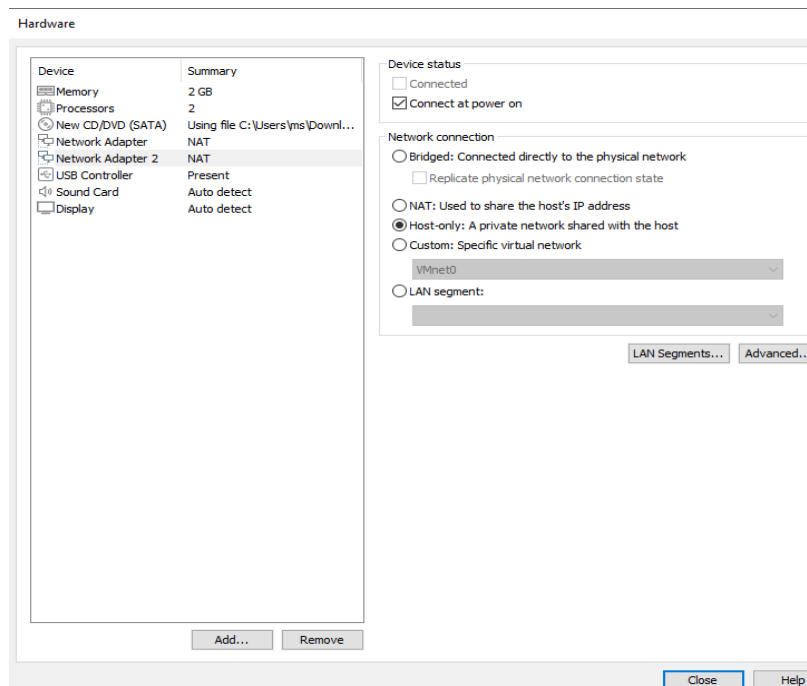
While working on this, I learned that not every protocol shows up just from normal browsing. For ICMP, nothing appeared in my capture until I ran a manual ping command myself, which showed me that some traffic has to be generated on purpose to actually study it.

I also realized I had ARP a bit wrong at first. I thought it was about finding an IP address, but it is actually the opposite: ARP takes a known IP and finds the matching MAC address for it. Going back and checking my own capture against my explanation helped me catch this and actually understand it properly, instead of just repeating a definition.

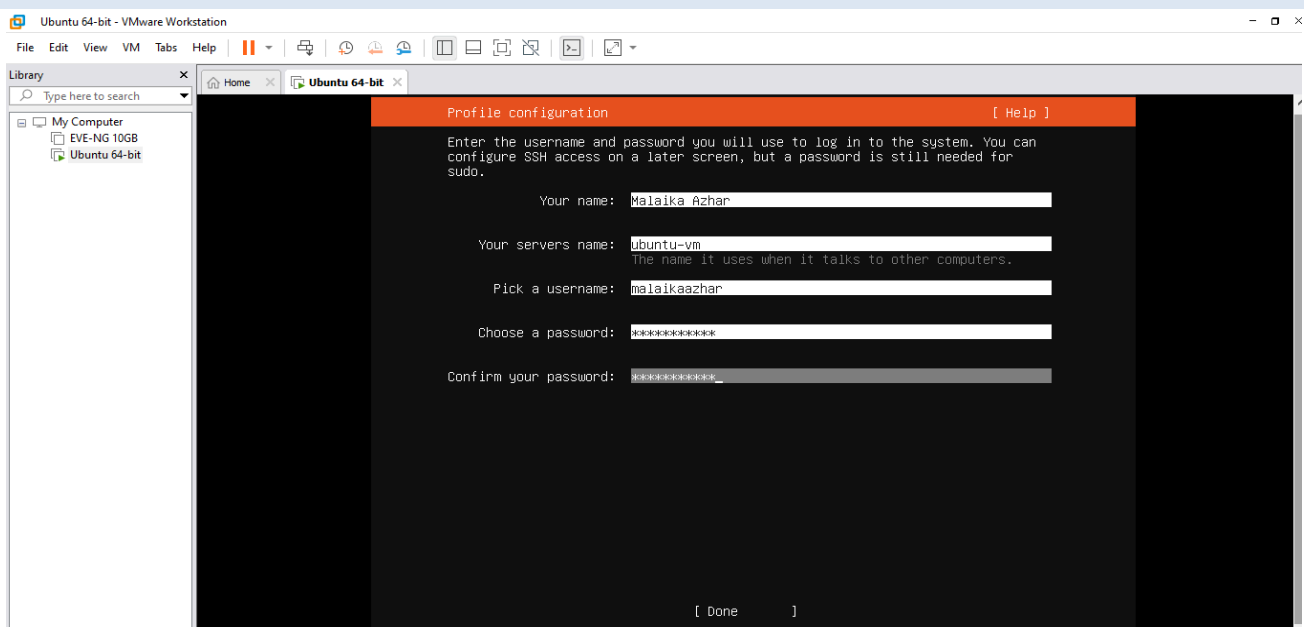
Day 2 — VM Lab Setup

1. VM Creation and Hardware Configuration

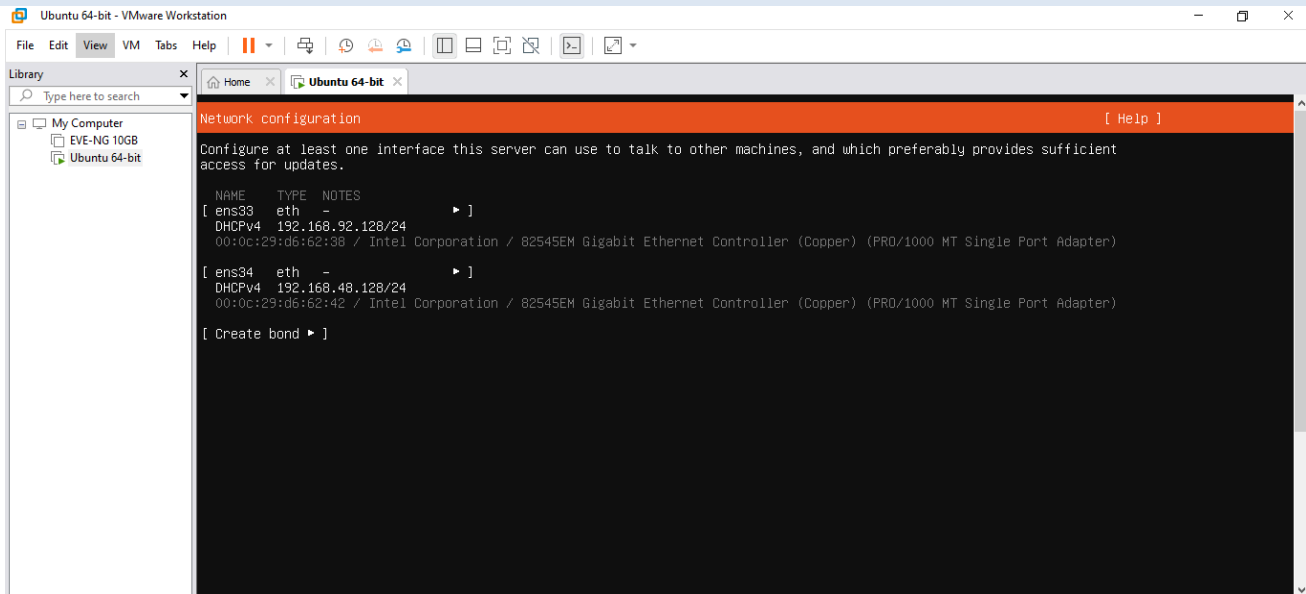
Created a new VM in VMware Workstation using the Ubuntu Server 22.04.5 LTS ISO. Allocated 2048 MB (2GB) RAM and 2 CPU cores. Configured two network adapters: one on NAT (internet access), one on Host-only (internal lab network).



2. Profile Configuration

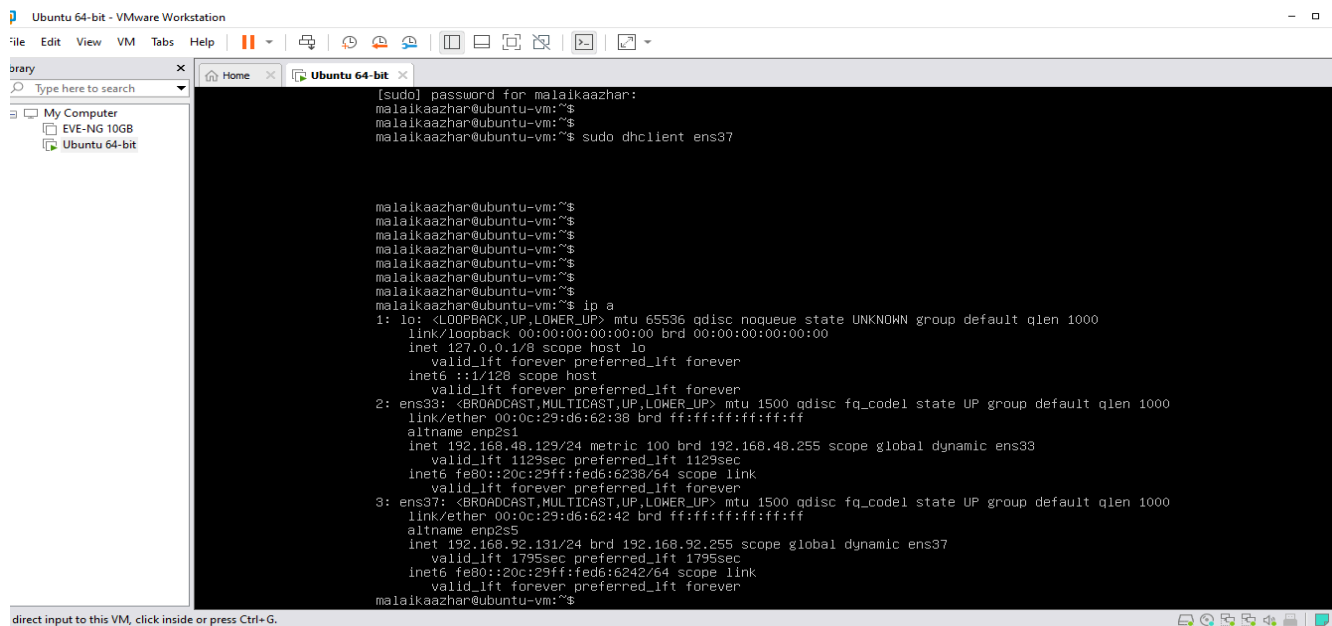


3. Network Configuration During Install



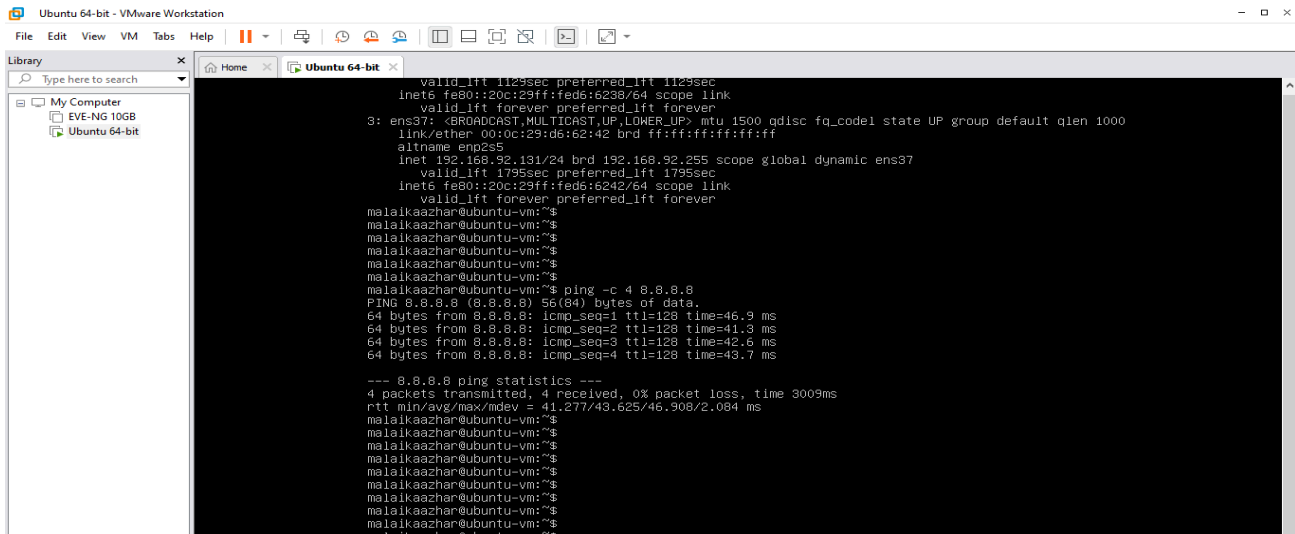
4. IP Address Confirmation (Post-Install)

After rebuilding the VM once (see Lessons Learned), only one interface initially appeared in `ip a`. The second adapter had to be re-added in VMware settings, then manually brought up inside Ubuntu with `sudo ip link set ens37 up` and `sudo dhclient ens37`. After that fix, both interfaces showed correctly: one on the NAT range (192.168.48.x) and one on the Host-only range (192.168.92.x). Note: VMware's default Host-only range differs from VirtualBox's typical 192.168.56.x — expected, not an error, since the brief assumes VirtualBox.



5. Connectivity Test

Ran `ping -c 4 8.8.8.8` to confirm internet access through the NAT adapter. Result: 4 packets transmitted, 4 received, 0% packet loss.

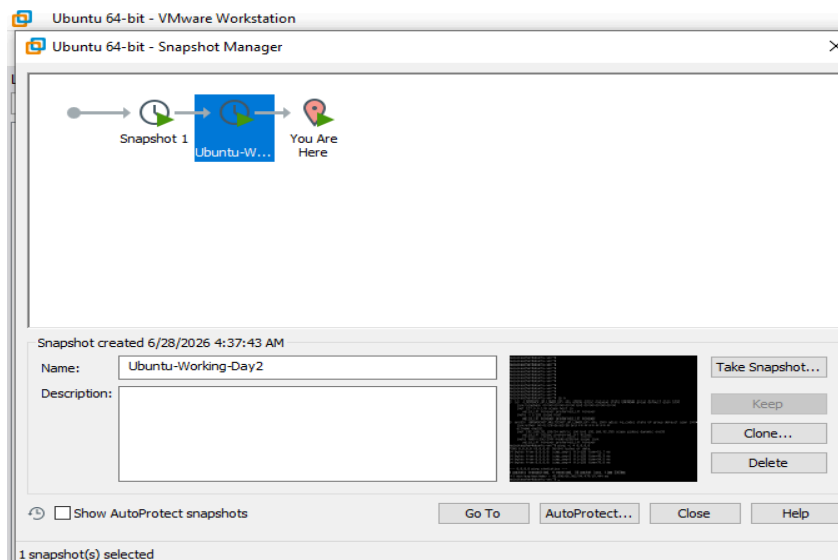


```
valid_lft 1129sec preferred_lft 1129sec
inet6 fe80::20c:29ff:fed6:6242/64 scope link
    valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:d6:62:42 brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    inet 192.168.92.131/24 brd 192.168.92.255 scope global dynamic ens37
        valid_lft 1795sec preferred_lft 1795sec
    inet6 fe80::20c:29ff:fed6:6242/64 scope link
        valid_lft forever preferred_lft forever
malaikaazhar@ubuntu-vm:~$ 
malaikaazhar@ubuntu-vm:~$ 
malaikaazhar@ubuntu-vm:~$ 
malaikaazhar@ubuntu-vm:~$ 
malaikaazhar@ubuntu-vm:~$ 
malaikaazhar@ubuntu-vm:~$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=46.9 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=41.3 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=128 time=42.6 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=128 time=43.7 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3009ms
rtt min/avg/max/mdev = 41.277/43.625/46.908/2.084 ms
malaikaazhar@ubuntu-vm:~$ 
malaikaazhar@ubuntu-vm:~$ 
malaikaazhar@ubuntu-vm:~$ 
malaikaazhar@ubuntu-vm:~$ 
malaikaazhar@ubuntu-vm:~$ 
malaikaazhar@ubuntu-vm:~$
```

6. Snapshot

Took a snapshot named `Ubuntu-Working-Day2` once the above was confirmed working, as a rollback point before Wazuh installation.



7. What I Learned — Day 2

I mixed up mounting the ISO file with pointing VMware directly to it — mounting created a virtual DVD drive instead of just letting VMware read the file, and I had to eject it and select the file directly through VMware's own browse dialog instead.

I also ended up with two VMs by accident after a confused rebuild, and had to delete both through File Explorer (since right-click delete inside VMware didn't remove the underlying files) before starting clean with one VM.

The Ubuntu Server installer turned out to be keyboard-only — Tab and Enter, no mouse — which wasn't obvious to me at first and made the install feel stuck when it was actually just waiting for keyboard input.

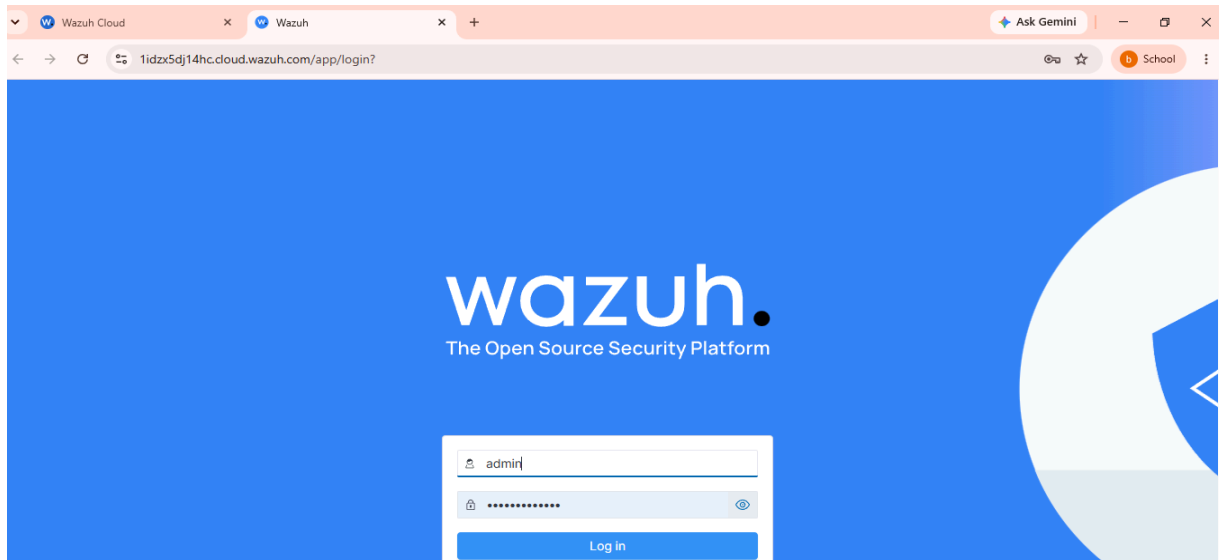
The VM also ran slowly on 2GB RAM, which made some screens look frozen for a minute or two when they were actually still processing.

Separately, I was given a command to install Wazuh that turned out to be a fake placeholder (echo "exit 0" > wazuh-install.sh) rather than the real installer. I caught this before it caused confusion, since running it produced no actual install output.

The most important fix was the missing second network adapter after the VM rebuild — I learned that VMware doesn't always preserve adapter configuration through a rebuild, and that a newly added adapter may need to be manually brought up inside Linux (ip link set up, dhclient) before it picks up an address, even after it's correctly configured on the VMware side.

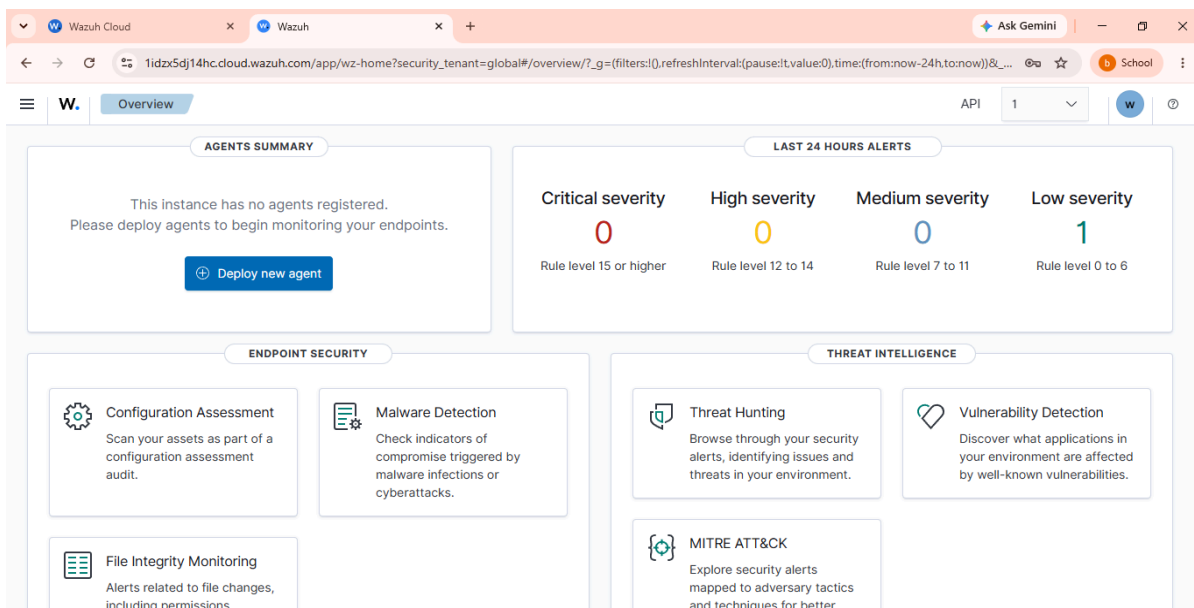
removing the RAM bottleneck entirely while keeping my Ubuntu VM as the monitored endpoint. This is a deliberate substitution for the local install, not a separate unrelated tool.

Note on static IP: the brief asks for a static IP on the Ubuntu Server VM before installing, since a self-hosted Manager needs a fixed address for agents to point at. Since the Manager is hosted on Wazuh Cloud rather than locally, agents point at the cloud's domain instead of a local IP, so this specific requirement doesn't directly apply the same way it would for a fully local setup.



2. Dashboard Login

Logged into the Wazuh Cloud dashboard using the admin account and the default credentials provided in the cloud console. At this point no agents were registered yet, confirming the cloud instance itself was live and reachable before any endpoint was connected.



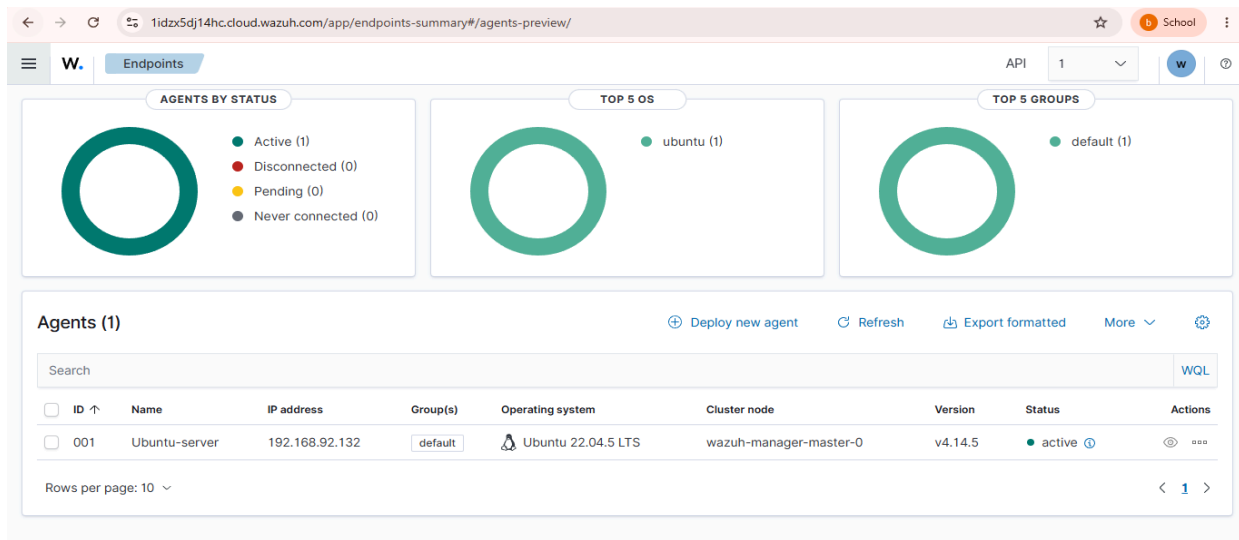
Day 5-6 — Agent Deployment and Alert Testing

1. Deploying the Agent

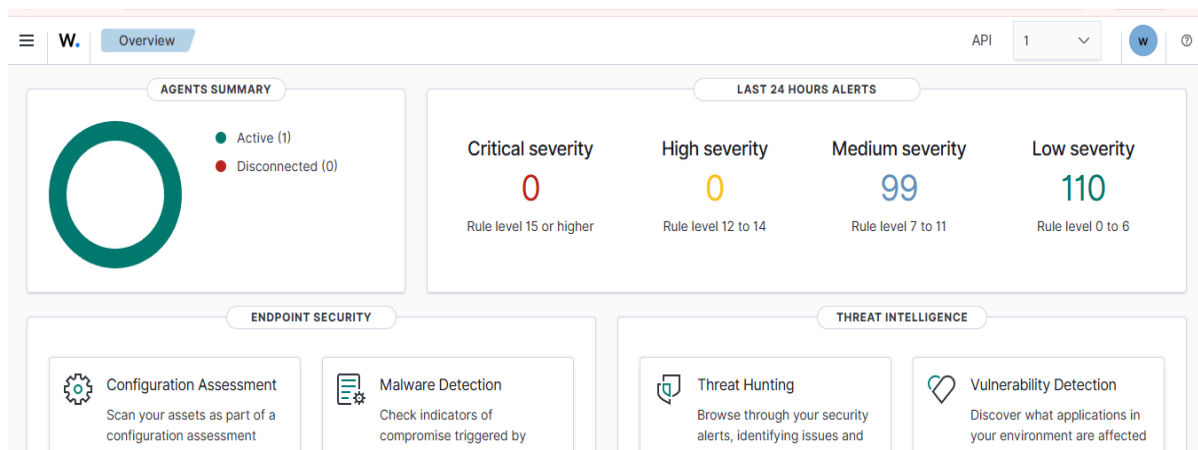
Used the dashboard's "Deploy new agent" wizard, selecting the DEB amd64 package to match the Ubuntu VM, and ran the generated install command on the VM. Brought the agent service up with:

- `sudo systemctl daemon-reload`
- `sudo systemctl enable wazuh-agent`
- `sudo systemctl start wazuh-agent`

The agent (Ubuntu-server, 192.168.92.132) registered successfully and shows as Active in the Endpoints list.



Once connected, the Overview dashboard began showing real telemetry from the endpoint — 99 medium and 110 low severity alerts within 24 hours, confirming the agent-to-cloud pipeline was actually working, not just showing as connected.



2. Deploying a Second Agent — Windows Host

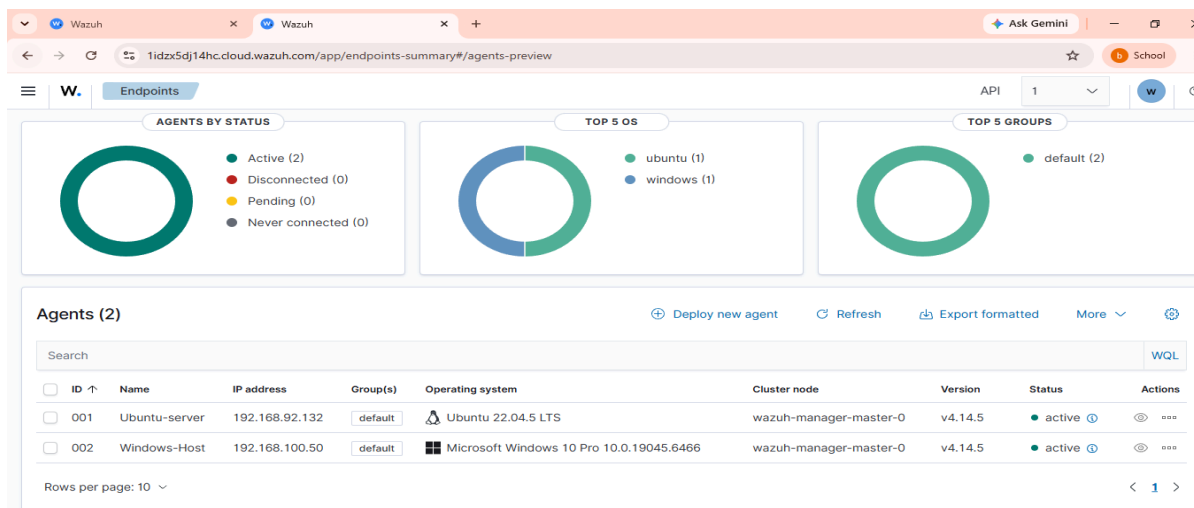
The brief requires two connected agents. Since the Windows 10 VM was skipped per the approved hardware plan, I deployed the agent directly on my host Windows machine instead — using the dashboard's Windows deployment wizard to generate a PowerShell install command, run as Administrator:

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/powershell

PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> >> Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.5-1.msi -OutFile $env:temp\wazuh-agent.msi; msisexec.exe /i $env:temp\wazuh-agent.msi /q WAZUH_MANAGER="1idx5dj14hc.cloud.wazuh.com" WAZUH_REGISTRATION_PASSWORD="WzQyY7p5Ym4zgYTzHx6vVxK2r0igxCKTO" WAZUH_AGENT_NAME="Windows-Host"
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> >> NET START WazuhSvc
The Wazuh service was started successfully.
PS C:\WINDOWS\system32>
```

Both agents now show Active at the same time — Ubuntu-server and Windows-Host.



3. Agent Configuration (ossec.conf)

Reviewed the Ubuntu agent's local file monitoring configuration through the dashboard's Endpoints > Configuration > Logs view, confirming the active log paths being monitored (/var/ossec/logs/active-responses.log, /var/log/dpkg.log) and log format (syslog).

1idzx5dj14hc.cloud.wazuh.com/app/endpoints-summary#/agents?tab=configuration&agent=001

Endpoints Ubuntu-server Configuration API 1

Logs Journald Commands Sockets

Logs files
List of log files that will be analyzed

- /var/ossec/logs/active-responses.log
- /var/log/dpkg.log

Log format: syslog

Log location: /var/ossec/logs/active-responses.log

Redirect output to this socket: agent

If the expression matches, the log will be ignored: -

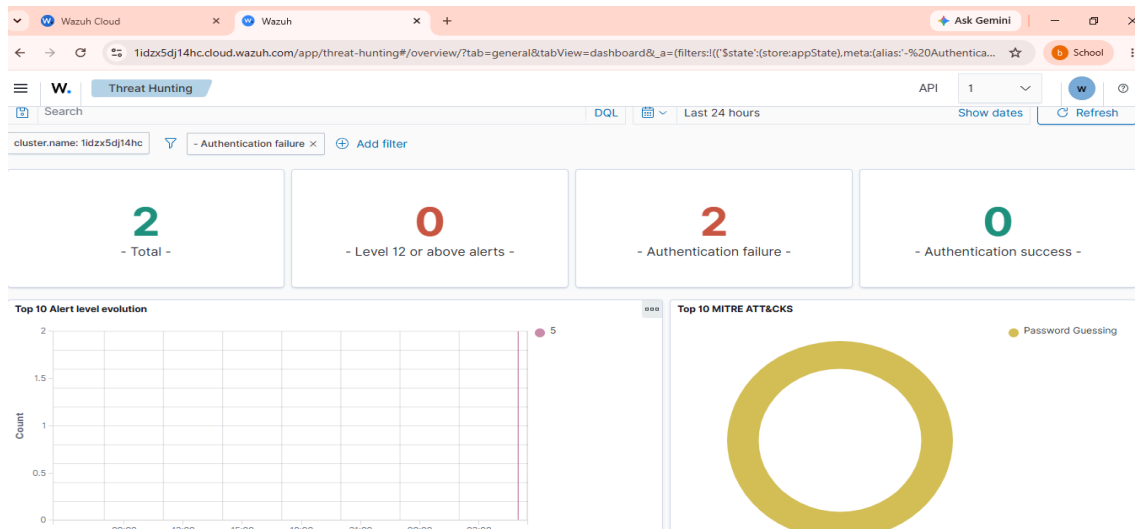
The log will only be processed if the expression matches: -

4. Generating a Real Failed-Login Alert

To confirm the pipeline could actually detect malicious-looking activity, I tried to generate a failed-authentication event. This took a few attempts to get right: `ssh invaliduser@localhost` was refused outright (no SSH session reached the auth layer), and `su invaliduser / su malaika` both failed with “user does not exist” rather than a real authentication failure, since neither was a valid local account. The attempt that actually worked was `su root` with a deliberately wrong password, which produced a genuine logged Authentication failure.

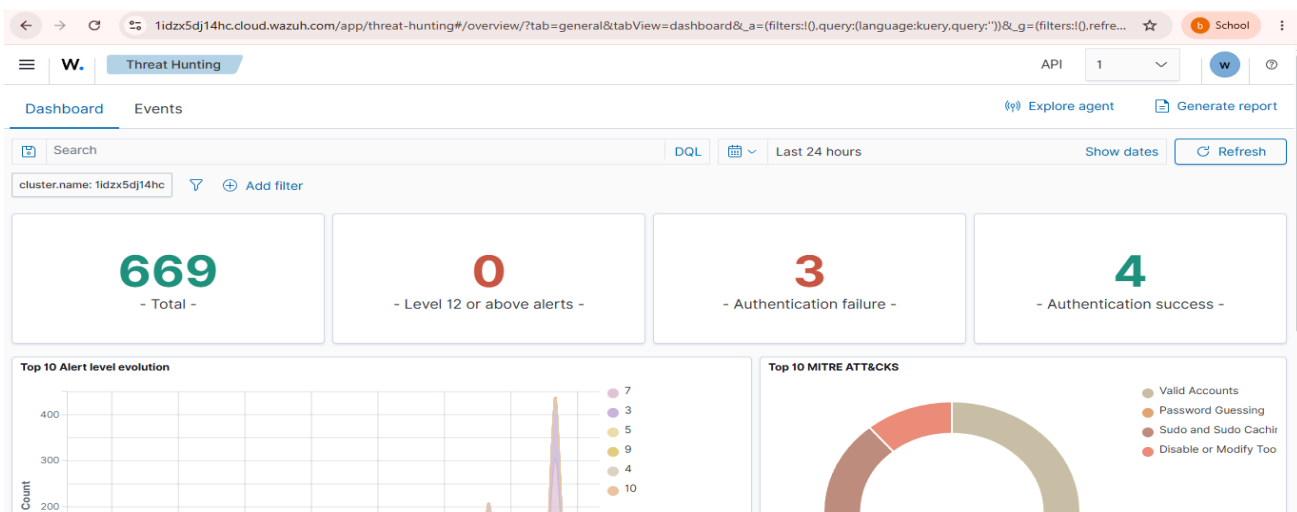
```
malaikaazhar@ubuntu2:~$ sudo systemctl daemon-reload
malaikaazhar@ubuntu2:~$ sudo systemctl enable wazuh-agent
Created symlink /etc/systemd/system/multi-user.target.wants/wazuh-agent.service → /lib/systemd/systemd-wazuh-agent.service.
malaikaazhar@ubuntu2:~$ sudo systemctl start wazuh-agent
malaikaazhar@ubuntu2:~$
malaikaazhar@ubuntu2:~$
malaikaazhar@ubuntu2:~$
malaikaazhar@ubuntu2:~$
malaikaazhar@ubuntu2:~$ ssh invaliduser@localhost
ssh: connect to host localhost port 22: Connection refused
malaikaazhar@ubuntu2:~$ su invaliduser
su: user invaliduser does not exist or the user entry does not contain all the required fields
malaikaazhar@ubuntu2:~$ su malaika
su: user malaika does not exist or the user entry does not contain all the required fields
malaikaazhar@ubuntu2:~$ su root
Password:
sssssssssu: Authentication failure
malaikaazhar@ubuntu2:~$
malaikaazhar@ubuntu2:~$
malaikaazhar@ubuntu2:~$
malaikaazhar@ubuntu2:~$
malaikaazhar@ubuntu2:~$
malaikaazhar@ubuntu2:~$
malaikaazhar@ubuntu2:~$ _
```

This showed up on the Wazuh Cloud side under Threat Hunting, filtered to Authentication failure: 2 events recorded, mapped under Top 10 MITRE ATT&CKs to Password Guessing — confirming the alert pipeline correctly classified the activity, not just logged it.



5. General Alerts View (Unfiltered)

Checked the general Threat Hunting overview with no filters applied, confirming the full pipeline across both agents: 669 total events in the last 24 hours.



6. What I Learned — Day 3-6

The biggest decision this stretch was choosing not to keep forcing the local install. Once the pre-flight check and the Indexer's memory requirement both pointed at the same hardware limit, switching to Wazuh Cloud's trial was a faster and more honest path than repeatedly fighting a 2GB VM.

Generating an actual failed-login alert took more attempts than expected — trying invalid or non-existent usernames doesn't produce the same kind of event as a real authentication

failure against a valid account. Working through that difference by trial and error was a more useful lesson than getting it right on the first try.

Day 7 — Dashboard Navigation and Alert Triage

Spent time in the Threat Hunting Events view (general view, not the File Integrity Monitoring-only tab, which only ever shows one rule type) to find a genuine spread of alert types from the Windows-Host agent.

Also reviewed the Rules management page to understand where these rule IDs and levels actually come from — Wazuh ships with 4,513 built-in rules, organized by group (syslog, firewall, windows, wazuh, etc.), each with its own severity level baked in by default.

ID ↑	Description	Groups	Level	File	Path
1	Generic template for all syslog rules.	syslog	0	0010-rules_config.xml	ruleset/rules
2	Generic template for all firewall rules.	firewall	0	0010-rules_config.xml	ruleset/rules
3	Generic template for all ids rules.	ids	0	0010-rules_config.xml	ruleset/rules
4	Generic template for all web rules.	web-log	0	0010-rules_config.xml	ruleset/rules
5	Generic template for all web proxy rules.	squid	0	0010-rules_config.xml	ruleset/rules
6	Generic template for all windows rules.	windows	0	0010-rules_config.xml	ruleset/rules
7	Generic template for all wazuh rules.	ossec	0	0010-rules_config.xml	ruleset/rules
200	Grouping of wazuh rules.	wazuh	0	0016-wazuh_rules.xml	ruleset/rules
201	Agent event queue rule	agent_flooding, wazuh	0	0016-wazuh_rules.xml	ruleset/rules
202	Agent event queue is level full.	agent_flooding, wazuh	7	0016-wazuh_rules.xml	ruleset/rules

Raw evidence from the Events view, before being organized into the table below:

timestamp	agent.name	rule.description	rule.level	rule.id
Jun 29, 2026 @ 14:16:44.3...	Windows-Host	Service startup type was changed	3	61104
Jun 29, 2026 @ 14:14:46.3...	Windows-Host	Integrity checksum changed.	7	550
Jun 29, 2026 @ 14:14:46.0...	Windows-Host	Integrity checksum changed.	7	550
Jun 29, 2026 @ 14:14:43.3...	Windows-Host	File deleted.	7	553
Jun 29, 2026 @ 14:08:51.7...	Windows-Host	Integrity checksum changed.	7	550
Jun 29, 2026 @ 14:08:51.4...	Windows-Host	Integrity checksum changed.	7	550
Jun 29, 2026 @ 14:08:46.5...	Windows-Host	File added to the system.	5	554
Jun 29, 2026 @ 14:08:46.3...	Windows-Host	Integrity checksum changed.	7	550
Jun 29, 2026 @ 14:08:43.3...	Windows-Host	File deleted.	7	553
Jun 29, 2026 @ 14:06:43.4...	Windows-Host	Integrity checksum changed.	7	550
Jun 29, 2026 @ 14:06:43.0...	Windows-Host	Integrity checksum changed.	7	550
Jun 29, 2026 @ 14:06:40.5...	Windows-Host	File added to the system.	5	554
Jun 29, 2026 @ 13:59:48.3...	Windows-Host	Service startup type was changed	3	61104
Jun 29, 2026 @ 13:57:52.6...	Windows-Host	Integrity checksum changed.	7	550
Jun 29, 2026 @ 13:57:52.0...	Windows-Host	Integrity checksum changed.	7	550

	Time	System Name	Description	Severity	Alert ID
	Jun 29, 2026 @ 13:57:47.2...	Windows-Host	File deleted.	7	553
	Jun 29, 2026 @ 13:51:57.9...	Windows-Host	Integrity checksum changed.	7	550
	Jun 29, 2026 @ 13:51:53.1...	Windows-Host	File added to the system.	5	554
	Jun 29, 2026 @ 13:51:52.9...	Windows-Host	Integrity checksum changed.	7	550
	Jun 29, 2026 @ 13:51:48.9...	Windows-Host	File deleted.	7	553
	Jun 29, 2026 @ 13:49:48.2...	Windows-Host	Integrity checksum changed.	7	550
	Jun 29, 2026 @ 13:49:47.2...	Windows-Host	Integrity checksum changed.	7	550
	Jun 29, 2026 @ 13:49:44.1...	Windows-Host	File added to the system.	5	554
	Jun 29, 2026 @ 13:49:40.0...	Windows-Host	Service startup type was changed	3	61104
	Jun 29, 2026 @ 13:46:22.9...	Windows-Host	Integrity checksum changed.	7	550
	Jun 29, 2026 @ 13:46:22.3...	Windows-Host	Integrity checksum changed.	7	550
	Jun 29, 2026 @ 13:46:22.1...	Windows-Host	Software protection service scheduled successfully.	3	60642
	Jun 29, 2026 @ 13:45:35.9...	Windows-Host	The database engine attached a database.	3	60798
	Jun 29, 2026 @ 13:45:35.7...	Windows-Host	The database engine has completed recovery steps.	3	60809
	Jun 29, 2026 @ 13:45:35.4...	Windows-Host	The database engine is replaying log file C:\Winnt\system32\wins\j50.log.	3	60808

Rows per page: 15

< 1 2 3 4 5 ... 62 >

	Time	System Name	Description	Severity	Alert ID
	Jun 29, 2026 @ 13:45:34.6...	Windows-Host	The database engine is initiating recovery steps.	3	60807
	Jun 29, 2026 @ 13:45:33.7...	Windows-Host	The database engine is starting a new instance.	3	60805
	Jun 29, 2026 @ 13:43:15.5...	Windows-Host	Service startup type was changed	3	61104
	Jun 29, 2026 @ 13:41:27.1...	Windows-Host	Integrity checksum changed.	7	550
	Jun 29, 2026 @ 13:41:26.9...	Windows-Host	Integrity checksum changed.	7	550
	Jun 29, 2026 @ 13:41:26.7...	Windows-Host	Integrity checksum changed.	7	550
	Jun 29, 2026 @ 13:41:26.5...	Windows-Host	Integrity checksum changed.	7	550
	Jun 29, 2026 @ 13:41:26.2...	Windows-Host	Integrity checksum changed.	7	550
	Jun 29, 2026 @ 13:41:15.3...	Windows-Host	File deleted.	7	553
	Jun 29, 2026 @ 13:41:07.8...	Windows-Host	Software protection service scheduled successfully.	3	60642
	Jun 29, 2026 @ 13:41:07.6...	Windows-Host	Integrity checksum changed.	7	550
	Jun 29, 2026 @ 13:40:37.5...	Windows-Host	Windows application error event.	9	60602
	Jun 29, 2026 @ 13:40:37.5...	Windows-Host	Integrity checksum changed.	7	550
	Jun 29, 2026 @ 13:40:37.5...	Windows-Host	Integrity checksum changed.	7	550
	Jun 29, 2026 @ 13:40:15.1...	Windows-Host	Integrity checksum changed.	7	550

Below are the 10 most frequent distinct alert types found across these pages, each with my own assessment.

Rule	Level	Description	My Assessment
550	7	Integrity checksum changed (by far the most frequent event)	Routine — FIM flagging file content changes. Fires constantly, likely tied to logs/system files that update on their own. Worth checking the exact file path before fully ruling out, but the repetition pattern looks like background noise, not an attack.
553	7	File deleted	Worth a closer look — file deletion is normally more notable than a modification, but given how often it appears alongside 550, likely part of routine log rotation/cleanup rather than tampering.
554	5	File added to the system	Routine — lower severity than deletion, consistent with normal software/log activity.

Rule	Level	Description	My Assessment
61104	3	Service startup type was changed	Low severity per Wazuh's own rating. Could be legitimate IT/admin config, but service-type changes can also be used to disable security tools — worth knowing which service before fully dismissing.
60602	9	Windows application error event	Real concern — the standout in this list. Level 9 is meaningfully higher than everything else here; application errors at this severity can indicate instability or an exploitation attempt. This is the one I'd investigate first.
60642	3	Software protection service scheduled successfully	Routine — Windows' own licensing/activation service, a normal background task.
60798	3	Database engine attached a database	Routine — part of a Windows service (likely Search/WMI) starting up.
60805	3	Database engine starting a new instance	Routine — same startup sequence as above.
60807	3	Database engine initiating recovery steps	Routine — normal recovery step after a service restart, not a sign of compromise.
60808	3	Database engine replaying log file (wins\l\j50.log)	Routine — final step of the same normal startup/recovery sequence.

What I Learned — Day 7

The File Integrity Monitoring tab only ever shows one rule type (550), which made it look at first like that was the only thing happening on the system. Switching to the general Threat Hunting Events view (not agent-specific, not FIM-specific) showed a much wider real picture — service changes, application errors, and a Windows database engine startup sequence, alongside the file integrity events.

Severity level alone doesn't tell the full story either — several Level 7 events here are routine background noise, while the single Level 9 event (the application error) stood out as the one actually worth investigating, exactly as the brief warned: frequency and context matter alongside the raw severity number.

Common Issues — What Actually Happened This Week

The brief lists known problem areas to watch for. Here's how each one actually played out in my environment:

Known Issue (Per Brief)	What Actually Happened
VM networking not behaving as expected	This happened, though on VMware rather than VirtualBox. After a VM rebuild, only one network adapter came back instead of two. Fixed by re-adding the missing adapter in VMware settings, then manually bringing it up inside Ubuntu with ip link set up and dhclient before it picked up an address.
Wazuh services failing due to insufficient memory	Hit the equivalent of this before installation even reached the Indexer stage — the installer's own pre-flight check flagged the 2GB VM as below the recommended minimum. Rather than force it through and risk the Indexer crashing as the brief warns, I used Wazuh Cloud instead, which avoids this limit entirely.
Agents showing Disconnected or Never Connected	Did not occur. Both agents (Ubuntu-server and Windows-Host) registered and showed Active on the first deployment attempt for each.
No alerts appearing after connecting agents	Did not occur. Telemetry began appearing almost immediately after each agent connected, and a deliberately triggered failed-login event was visible in the dashboard within minutes.
Dashboard certificate warning	Not observed in the same form the brief describes, since Wazuh Cloud serves the dashboard over a managed certificate rather than the self-signed certificate a fully local install would generate.

Closing Reflection — Week 1

The single biggest factor in this week wasn't any one technical concept — it was working within a hardware limit that the original brief didn't fully account for, and having to make real decisions about when to keep pushing on the original plan versus when to substitute a different but equally valid path.

Three substitutions ended up defining the week: VMware instead of VirtualBox, a single VM instead of three running simultaneously, and Wazuh Cloud instead of a local install. Each one was made for a specific, documented reason rather than as a shortcut, and each is recorded in this report alongside the evidence for why it was necessary.

Technically, the most useful lesson was that severity level and frequency don't always point the same direction — a Level 7 alert firing constantly turned out to be routine noise, while a single Level 9 event stood out precisely because it was rare. That distinction, mentioned directly in the Day 7 brief, only really became concrete once I was looking at my own real alert data instead of a textbook description of it.

Going into Week 2, the main outstanding item is the RAM upgrade — once that's done, the deferred Kali Linux VM and a fully local Wazuh install both become possible again, closing the gap between what the brief originally asked for and what this week's hardware allowed.